

Prueba técnica

AI Developer Engineer Semi-senior

PERFIL	TIEMPO SUGERIDO
Generative AI, Agentic Solutions & Cloud Development	48 a 72 horas

Objetivo de evaluación

Validar capacidad técnica, criterio de solución, claridad de comunicación y uso responsable de herramientas de desarrollo asistido por AI.

1. Objetivo de la prueba

Evaluar la capacidad del candidato para diseñar, construir y explicar una solución de AI aplicada a un caso empresarial realista, integrando Generative AI, RAG, agentes, APIs, cloud, seguridad, trazabilidad, evaluación y criterio arquitectónico.

La prueba debe permitir validar si el candidato puede pasar de un problema ambiguo a una solución funcional, simple, documentada y defendible ante un cliente.

2. Caso de negocio

Un cliente tiene una aplicación legada de gestion de solicitudes internas. Actualmente, los usuarios consultan manualmente documentos técnicos, procedimientos operativos y registros historicos para responder preguntas o tomar decisiones.

El cliente quiere construir un asistente inteligente que permita:

- Consultar documentación interna.
- Responder preguntas con base en fuentes cargadas.
- Ejecutar acciones simples mediante herramientas.
- Mantener trazabilidad de las interacciones.
- Ser extensible para integrarse luego con sistemas corporativos.
- Operar sobre AWS o Azure.

3. Reto técnico

Construir un Agente RAG empresarial con API backend, capacidad de consulta documental, uso de herramientas y criterios básicos de production readiness.

Opción de plataforma

El candidato puede elegir AWS, Azure o ejecución local documentada. Si usa ejecución local, debe explicar como desplegaría la solución en AWS o Azure.

4. Rutas técnicas sugeridas

Ruta AWS	Ruta Azure
Amazon Bedrock o modelo equivalente	Azure OpenAI o modelo equivalente
S3 o almacenamiento local simulado	Azure AI Search, pgvector, Chroma, FAISS o vector store equivalente
DynamoDB o base de datos local	Azure Functions, Container Apps o ejecución local documentada
OpenSearch, pgvector, Chroma, FAISS o vector store equivalente	Cosmos DB o base de datos local
Lambda, ECS o ejecución local documentada	Application Insights o logging local estructurado
CloudWatch o logging local estructurado	

5. Alcance mínimo requerido

5.1 Backend

- API en Python usando FastAPI, Flask o framework similar.
- Endpoint para consultar al asistente.
- Endpoint para cargar o procesar documentos.
- Endpoint para consultar historial o logs básicos.
- Manejo básico de errores.
- Documentación de uso de la API.

5.2 RAG

- Ingesta de documentos.
- Chunking.
- Generación de embeddings.
- Almacenamiento en vector store.
- Recuperación de contexto.
- Respuesta generada con base en documentos.
- Inclusión de fuentes o fragmentos usados como referencia.

5.3 Agente con herramientas

Implementar al menos 2 herramientas que el agente pueda usar. Ejemplos:

- Consultar estado de una solicitud en un archivo JSON o base de datos mock.
- Crear un resumen ejecutivo de una solicitud.
- Clasificar una solicitud por prioridad.
- Generar una recomendación de modernización.
- Calcular esfuerzo estimado según reglas simples.
- Consultar una lista de servicios cloud sugeridos.

5.4 AI-CDL / AI-assisted development

El candidato debe usar o documentar como usaría herramientas de desarrollo asistido por AI como Kiro, Claude Code, OpenAI Codex, GitHub Copilot, Cursor o equivalentes.

- Qué tareas apoyo con AI.
- Qué validaciones realizo sobre el código generado.
- Qué riesgos identificó.
- Cómo evitó depender ciegamente de la herramienta.

5.5 Seguridad y operación

- Manejo de variables de entorno.
- Separacion de secretos.
- Logging básico.
- Validación básica de entradas.
- Manejo de errores.
- Identificación de riesgos como prompt injection, fuga de datos o respuestas no fundamentadas.
- Recomendaciones para llevar la solución a producción.

5.6 Evaluación de calidad

Debe incluir al menos 5 preguntas de prueba con pregunta, respuesta esperada o criterio de aceptación, resultado obtenido y observación del candidato.

- Metrica simple de precision, deseable.
- Evaluación de groundedness, deseable.
- Prueba de respuesta cuando no existe información suficiente.
- Prueba de prompt injection.

6. Entregables esperados

- Repositorio de código.
- README con instrucciones de instalación y ejecución.
- Diagrama de arquitectura.
- Documento corto de decisiones técnicas.
- Evidencia de pruebas ejecutadas.
- Video corto de máximo 7 minutos mostrando la solución.
- Sección de mejoras futuras.
- Sección de riesgos y consideraciones para producción.

7. Criterios de evaluación

Criterio	Peso
Diseño de arquitectura y simplicidad de la solución	15%
Calidad del backend, API y estructura del código	15%
Implementación RAG	20%
Diseño del agente y uso de herramientas	15%
Seguridad, logging, trazabilidad y readiness para producción	15%
Evaluación de respuestas y pruebas	10%

Documentación, comunicación y claridad de explicación	10%
---	-----